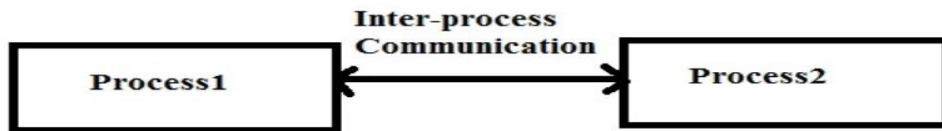


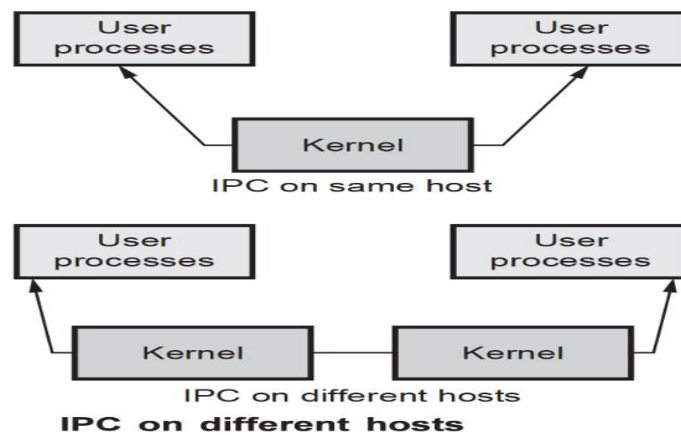
Inter Process Communication (IPC) using Pipes

Inter Process Communication (IPC) is a mechanism that involves communication of one process with another process.



Communication can be of two types

- Between related processes initiating from only one process, such as parent and child processes.
- Between unrelated processes, or two or more different processes.



Need for IPC

- **Sharing of information:** Inter-process communication helps in sharing information when several processes try to access a particular file concurrently. It is, therefore, necessary for the processes to cooperate with each other so that the information provided is correct.
- **Speed:** Dividing a big task into smaller tasks and then running them concurrently can speed up the entire functioning of the system. We need inter-process communication in such a case. The multiprocessor environment is a very good example of this.
- **Modularity:** When a system is divided into modules, such modules may require information sharing.
- **Convenience:** Allowing multiple processes to run simultaneously provides a better experience to any user.

In inter-process communication (IPC), pipes are a fundamental mechanism that allows processes to communicate data with each other. One of the ways to achieve Inter Process Communication is through pipes. It is nothing but a mechanism using which the output of a single process is directed into the input of another process.

Advantages:

1. **Simplicity:** Pipes are a simple and straightforward way for processes to communicate with each other.
2. **Efficiency:** Pipes are an efficient way for processes to communicate, as they can transfer data quickly and with minimal overhead.
3. **Reliability:** Pipes are a reliable way for processes to communicate, as they can detect errors in data transmission and ensure that data is delivered correctly.
4. **Flexibility:** Pipes can be used to implement various communication protocols, including one-way and two-way communication.

Disadvantages:

1. **Limited capacity:** Pipes have a limited capacity, which can limit the amount of data that can be transferred between processes at once.
2. **Unidirectional:** In a unidirectional pipe, only one process can send data at a time, which can be a disadvantage in some situations.
3. **Synchronization:** In a bidirectional pipe, processes must be synchronized to ensure that data is transmitted in the correct order.
4. **Limited scalability:** Pipes are limited to communication between a small number of processes on the same computer, which can be a disadvantage in large-scale distributed systems

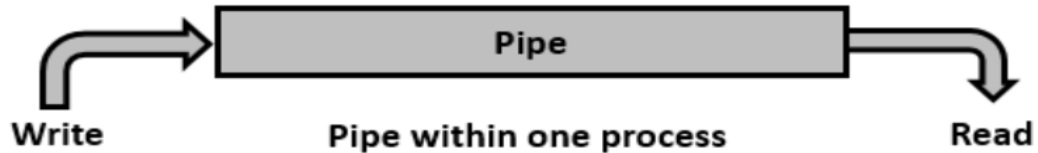
Pipes can be categorized into two types: unnamed (or anonymous) pipes and named pipes.

Unnamed Pipes or Traditional Pipes

- Unnamed pipes provide a way for processes to communicate in a unidirectional fashion, meaning data flows in only one direction. Unnamed pipes are used when communication needs to occur between a parent and its child process. The lifetime of an unnamed pipe is usually tied to the duration of the processes using it; it does not persist after the processes .

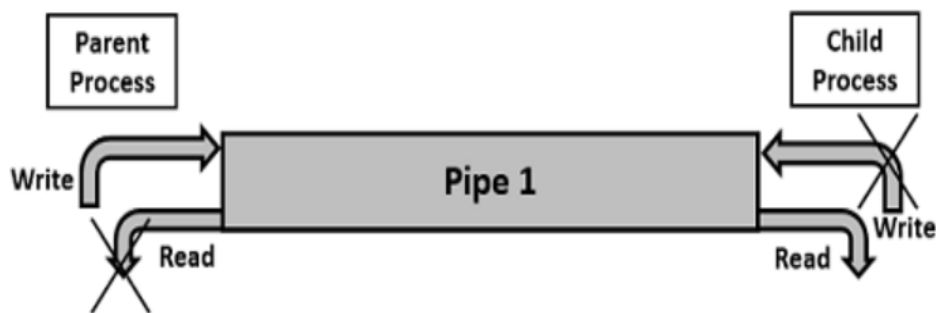
A usual pipe file has an input end and an output end. We can read from the output end and write into a pipe from the input end. There is an array of two pointers present in a pipe descriptor. One

pointer is for storing the input end of the pipe and the other pointer is for storing the output end of the pipe.



Let us assume that two processes, Process One and Process Two, want to communicate with each other using pipes.

- The first Process should keep its write end of the pipe open and the read end closed.
- Process Two should keep its read end open and close its write end. On successful creation, a pipe is given a fixed size in bytes.



- If the pipe is not full and some process wants to write into the pipe, the write request is executed spontaneously. However, if the pipe is full when the process wants to write into the pipe, it is blocked until the state of the pipe changes.
- The same things happen if a reading process tries to read more bytes than what is currently in the pipe and gets blocked. On the other hand, the reading process is executed without any issues if it tries to read-only sufficient bytes that are there in the pipe. Only a single process can access a pipe at any particular point in time.

System Calls

- **int pipe (int fd[2]);** The input parameter is an array of two file descriptors fd[0] and fd[1]. A file descriptor is in fact an integer value. The system call returns a -1 in case of a failure. If the call is successful, it will return two integer values which are file descriptors fd[0]&fd[1]. In pipes, fd[0] and fd[1] are used for reading and writing, respectively.
- **Write(fd,buf,count)**

- **fd**: The file descriptor to which data is to be written. When using unnamed pipes, this is typically the write-end of the pipe.
- **buf**: A pointer to the buffer containing the data to be written.
- **count**: The number of bytes to write from the buffer.
- **Return Value**
- On success, **write()** returns the number of bytes written. This number may be less than **count** if there is insufficient space available on the pipe.
- On error, -1 is returned, and **errno** is set to indicate the error.

Read(fd,buf,count):

- **fd**: The file descriptor from which data is read. For unnamed pipes, this is typically the read end.
- **buf**: A pointer to the buffer where the read data should be stored.
- **count**: The maximum number of bytes to read.
- **Return Value**
- On successful completion, **read()** returns the number of bytes actually read and placed in the buffer. This value might be less than the number requested for several reasons, such as if fewer bytes are actually available at the moment in the pipe buffer.
- A return value of 0 indicates end-of-file (EOF); for a pipe, this means that no more data can be read because all file descriptors referring to the write end of the pipe have been closed.
- On error, -1 is returned, and **errno** is set to indicate the error.

Sample Program

```
#include<stdio.h>
#include<unistd.h>
#include<stdlib.h>
int main()
{
    int fd[2],n;
    char buffer[100];
    pid_t p;
    pipe(fd); //creates a unidirectional pipe with two end fd[0] and fd[1]
    p=fork();
    if(p>0) //parent
    {
        Close(fd[0]);
        printf("Parent Passing value to child\n");
        write(fd[1],"hello\n",6); //fd[1] is the write end of the pipe
        wait();
    }
```

```

}
else // child
{
close(fd[1]);
printf("Child printing received value\n");
n=read(fd[0],buffer,100); //fd[0] is the read end of the pipe
write(1,buffer,n);
}
}

```

FIFO

- In computing, a named pipe (also known as a **FIFO**) is one of the methods for inter-process communication. It is an extension to the traditional pipe concept on Unix.
- A named pipe, however, can last as long as the system is up, beyond the life of the process. It can be deleted if no longer used.
- Usually a named pipe appears as a file and generally processes attach to it for inter-process communication. A FIFO file is a special kind of file on the local storage which allows two or more processes to communicate with each other by reading/writing to/from this file.
- A FIFO special file is entered into the filesystem by calling *mkfifo()* in C. Once we have created a FIFO special file in this way, any process can open it for reading or writing, in the same way as an ordinary file. However, it has to be open at both ends simultaneously before you can proceed to do any input or output operations on it.
- The entire IPC process will consist of three programs:

Program1: to create a named pipe

Program2: process that will write into the pipe (sender process)

Program3: process that will receive data from pipe (receiver process)

//Program1: to create a named pipe

```

#include<stdio.h>
#include<sys/types.h>
#include<sys/stat.h>
int main()
{
    int res;
    res = mkfifo("fifo1",0777); //creates a named pipe with the name fifo1
    printf("named pipe created\n");
}
//Now compile and run this program.

```

//Program2: Writing to a fifo/named pipe (sender.c)

```

#include<unistd.h>
#include<stdio.h>
#include<fcntl.h>
int main()
{
    int res,n;
    res=open("fifo1",O_WRONLY);
    write(res,"Message",7);
    printf("Sender Process %d sent the data\n",getpid());
}

```

- **//Program 3:** Reading from the named pipe (receiver.c)

- #include<unistd.h>
- #include<stdio.h>

```

#include<fcntl.h>
int main()
{
    int res,n;
    char buffer[100];
    res=open("fifo1",O_RDONLY);
    n=read(res,buffer,100);
    printf("Reader process %d started\n",getpid());
    printf("Data received by receiver %d is: %s\n",getpid(), buffer);
}

```

The major differences between named and unnamed pipes are:-

1. As suggested by their names, a named type has a specific name which can be given to it by the user. Named pipe is referred through this name only by the reader and writer. All instances of a named pipe share the same pipe name. On the other hand, unnamed pipes are not given a name. It is accessible through two file descriptors that are created through the function `pipe(fd[2])`, where `fd[1]` signifies the write file descriptor, and `fd[0]` describes the read file descriptor.
2. An unnamed pipe is only used for communication between a child and its parent process, while a named pipe can be used for communication between two unnamed processes as well. Processes of different ancestry can share data through a named pipe.
3. A named pipe exists in the file system. After input-output has been performed by the sharing processes, the pipe still exists in the file system independently of the process, and can be used for communication between some other processes. On the other hand, an unnamed pipe vanishes as soon as it is closed, or one of the process (parent or child) completes execution.

4. Named pipes can be used to provide communication between processes on the same computer or between processes on different computers across a network, as in case of a distributed system. Unnamed pipes are always local; they cannot be used for communication over a network.
5. A Named pipe can have multiple process communicating through it, like multiple clients connected to one server. On the other hand, an unnamed pipe is a one-way pipe that typically transfers data between a parent process and a child process.

Semaphores: **semget**, **semop**, **semctl**,

Linux provides a **Semaphore API**, which has a set of functions that allow developers to work with semaphores in linux efficiently with their programs. Let us explore the key functions of the API,

semget

The **semget()** function is used to create a **new semaphore** or get an existing semaphore's identifier.

Syntax:

```
int semget(key_t key, int num_sems, int sem_flags);
```

It takes three arguments:

- **key:**

Used to identify the semaphore set. It can be either `IPC_PRIVATE` (to create a new semaphore set) or an identifier used to get an existing semaphore set.

- **num_sems:**

Number of semaphores to create in the semaphore set. Typically, this is set to 1 for single semaphores.

- **sem_flags:**

Permission flags for the semaphore linux set, specified as an octal number. It determines the read, write, and execute permissions for the semaphore set.

It returns the linux semaphore identifier (semid) if successful or -1 on failure.

semctl()

The **semctl()** function is used to control semaphore behavior.

Syntax:

```
int semctl(int semid, int sem_num, int command, ...);
```

It takes three arguments:

- **semid:**
Semaphore linux identifier representing the semaphore set on which the operation will be performed.
- **sem_num:**
Semaphore number within the semaphore set on which the operation will be performed. For single semaphores, this is usually set to 0.
- **command:**
Specifies the operation to be performed on the Semaphore linux set. The most used values are:
 - **IPC_STAT:**
Get the semaphore set status and store it in a struct semid_ds.
 - **IPC_SET:**
Set the semaphore set status using the information from a struct semid_ds.
 - **IPC_RMID:**
Remove the semaphore set from the system.
 - **GETVAL:**
Get the value of the semaphore.
 - **GETPID:**
Get the process ID of the last operation that modified the semaphore.

- **Optional Argument:**

Depending on the command used, there might be an optional fourth argument, which is a pointer to a union semun. The union semun allows specifying additional parameters for some commands, such as the new value for SETVAL or an array of semaphore values for SETALL.

This function returns the result of the specified command, depending on the operation performed. On failure, it returns -1 to indicate an error.

semop()

The **semop()** function is used to perform operations, such as locking and unlocking semaphores in linux.

Syntax:

```
int semop(int semid, struct sembuf *sops, size_t num_sops);
```

It takes three arguments:

- **semid:** The linux semaphore identifier.

- **sops:** Pointer to an array of struct sembuf, specifying the Semaphore linux operations to be performed.
- **num_sops:** Number of elements in the sops array, indicating the number of semaphore operations to be performed.

The struct sembuf data structure contains three fields:

- **sem_num:** Semaphore number within the semaphore set. For single semaphores, this is usually set to 0.
- **sem_op:** The operation to be performed on the semaphore, which can be a positive or negative value.
- **sem_flg:** Flags that control the behavior of the operation, such as SEM_UNDO (automatic semaphore release on process termination) or IPC_NOWAIT (return immediately if the semaphore is not available).

This function returns 0 if the semaphore operations are successful or -1, on error.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/sem.h>
int main() {
    int semaphore = semget(IPC_PRIVATE, 1, 0666 | IPC_CREAT);
    struct sembuf sem_op;

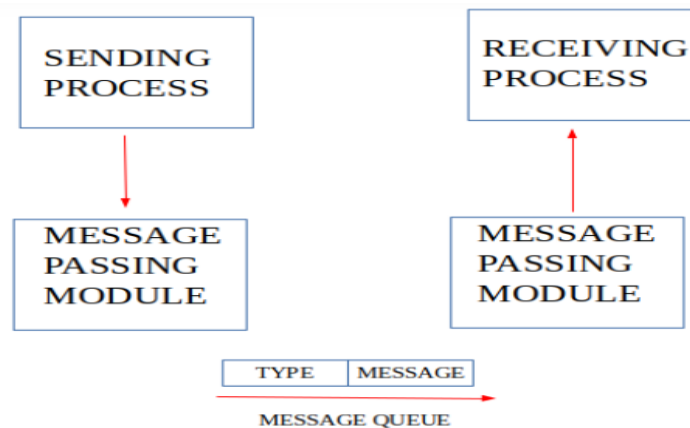
    if (semaphore < 0) {
        perror("Semaphore creation failed");
        exit(1);
    }
    printf("Semaphore created successfully.\n");
    sem_op.sem_num = 0;
    sem_op.sem_op = -1; // Lock the semaphore
    sem_op.sem_flg = 0;
    if (semop(semaphore, &sem_op, 1) == -1) {
        perror("Failed to lock semaphore");
        exit(1);
    }
    printf("Semaphore locked.\n");
    // Critical Section - Access shared resources here
    printf("In critical section.\n");
    sem_op.sem_op = 1; // Unlock the semaphore
    if (semop(semaphore, &sem_op, 1) == -1) {
        perror("Failed to unlock semaphore");
        exit(1);
    }
}
```

```
}

printf("Semaphore unlocked.\n");
if (semctl(semaphore, 0, IPC_RMID) == -1) {
    perror("Failed to remove semaphore");
    exit(1);
}
printf("Semaphore removed.\n");
return 0;
}
```

Message Queue

A **message queue** is an inter-process communication (IPC) mechanism that allows processes to exchange data in the form of messages between two processes. . It allows processes to communicate asynchronously by sending messages to each other where the messages are stored in a queue, waiting to be processed, and are deleted after being processed



To perform communication using message queues, following are the steps –

Step 1 – Create a message queue or connect to an already existing message queue (`msgget()`)

Step 2 – Write into message queue (`msgsnd()`)

Step 3 – Read from the message queue (`msgrcv()`)

Step 4 – Perform control operations on the message queue (`msgctl()`)

msgget

Purpose: The **msgget** system call is used to obtain access to a message queue. If a new queue is to be created, **msgget** can also initialize it.

Syntax: The function is defined in C as follows:

```
int msgget(key_t key, int msgflg);
```

key_t key: This is an identifier for the message queue. It can be specified explicitly by the programmer, or **IPC_PRIVATE** can be used to generate a unique key.

int msgflg: These flags determine the action to take if the message queue already exists or doesn't exist. It can also include permissions for the queue. Common flags include **IPC_CREAT** (create the message queue if it does not exist), **IPC_EXCL** (ensure the message queue is being created for the first time), along with the usual permission bits (like **0666**).

Return Value: It returns the message queue identifier (a non-negative integer) on success. If it fails, it returns -1 and sets the **errno** variable to indicate the error.

msgsnd

Purpose: **msgsnd** sends a message to the message queue. It allows processes to communicate asynchronously by placing messages onto a queue, which can then be read by other processes.

Syntax: `int msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg);`

int msqid: This is the message queue identifier returned by **msgget**.

const void *msgp: This is a pointer to the message that needs to be sent. The message must start with a long integer that represents the message type, followed by the actual data.

size_t msgsz: This is the size of the message data in bytes, not including the message type.

int msgflg: These flags control what the function should do if the message queue is full. Common flags include **IPC_NOWAIT** which causes the function to return immediately if the message cannot be sent because the queue is full.

Return Value: On success, **msgsnd** returns 0. On failure, it returns -1

msgrcv

Purpose : **msgrcv** is used to receive messages from a queue. It can be configured to retrieve messages of specific types or simply fetch messages in a first-in, first-out (FIFO) order.

Syntax : `size_t msgrcv(int msqid, void *msgp, size_t msgsz, long msgtyp, int msgflg);`

- **int msqid:** This is the message queue identifier returned by **msgget**.
- ***void msgp:** This is a pointer to the message buffer into which the received message will be placed. This buffer should be structured to match the message being received, typically starting with a long integer that indicates the message type.
- **size_t msgsz:** This specifies the size of the message buffer, excluding the size of the message type field.
- **long msgtyp:** This parameter determines which message to receive from the queue:
 - If **msgtyp** is 0, the next message in the queue is received.
 - If **msgtyp** is greater than 0, the next message of that type is received.
 - If **msgtyp** is less than 0, the first message of the lowest type that is less than or equal to the absolute value of **msgtyp** is received.
- **int msgflg:** These flags determine how the function behaves if the desired message is not immediately available. Common flags include:
 - IPC_NOWAIT:** Return immediately if no message is available that matches the criteria.

MSG_EXCEPT: Used with a positive **msgtyp** to receive the next message in the queue that is not of type **msgtyp**.

MSG_NOERROR: If the message is larger than **msgsz**, truncate it instead of returning an error.

Return Value: On success, **msgrcv** returns the size of the received message. If it fails, it returns -1

msgctl

Purpose : **msgctl** is used to control or modify the properties of a message queue, or to remove a message queue entirely from the system.

Syntax : `int msgctl(int msqid, int cmd, struct msqid_ds *buf);`

- **int msqid:** This is the message queue identifier returned by **msgget**.
- **int cmd:** This command argument specifies the action to be performed on the message queue. Common commands include:

IPC_STAT: Copy information from the kernel data structure associated with **msqid** into the **msqid_ds** structure pointed to by **buf**.

IPC_SET: Set the message queue attributes using the information in the **msqid_ds** structure pointed to by **buf**.

IPC_RMID: Remove the message queue. This command immediately deletes the queue and all its messages.

- ***struct msqid_ds buf:** This is a pointer to an instance of **msqid_ds** structure which is used to store or modify message queue attributes. This structure is defined as follows:

```
struct msqid_ds {  
    struct ipc_perm msg_perm; // Ownership and permissions  
    time_t      msg_stime; // Time of last msgsnd  
    time_t      msg_rtime; // Time of last msgrcv  
    time_t      msg_ctime; // Time of last change  
    unsigned long msg_cbytes; // Current number of bytes in queue (non-standard)  
    msgqnum_t    msg_qnum; // Current number of messages in queue  
    msglen_t     msg_qbytes; // Maximum number of bytes allowed in queue  
    pid_t        msg_lspid; // PID of last msgsnd  
    pid_t        msg_lrpid; // PID of last msgrcv  
};
```

Return Value: On success, **msgctl** returns 0. If it fails, it returns -1

Sample Program

Sender Process

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>

struct msgbuf {
    long mtype; // message type must be > 0
    char mtext[200]; // message data
};

int main() {
    key_t key = 1234; // predefined key
    int msgid;
    struct msgbuf msg;
    // Create a message queue
    msgid = msgget(key, 0666 | IPC_CREAT);
    if (msgid < 0) {
        perror("msgget");
        exit(EXIT_FAILURE);
    }
    // Prepare a message to send
    msg.mtype = 1;
    strcpy(msg.mtext, "Hello, this is message 1");
    // Send the message
    if (msgsnd(msgid, &msg, strlen(msg.mtext), 0) < 0) {
        perror("msgsnd");
        exit(EXIT_FAILURE);
    }
    printf("Sent: %s\n", msg.mtext);
    // Prepare another message
    msg.mtype = 2;
```

```

strcpy(msg.mtext, "Hello, this is message 2");
// Send the message
if (msgsnd(msgid, &msg, strlen(msg.mtext), 0) < 0) {
    perror("msgsnd");
    exit(EXIT_FAILURE);
}
printf("Sent: %s\n", msg.mtext);
return 0;
}

```

Receiver Process

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>
struct msgbuf {
    long mtype; // message type must be > 0
    char mtext[200]; // message data
};
int main() {
    key_t key = 1234; // predefined key
    int msgid;
    struct msgbuf msg;
    // Access the message queue
    msgid = msgget(key, 0666 | IPC_CREAT);
    if (msgid < 0) {
        perror("msgget");
        exit(EXIT_FAILURE);
    }
    // Receive a message of any type
    if (msgrcv(msgid, &msg, sizeof(msg.mtext), 0, 0) < 0) {
        perror("msgrcv");
    }
}

```

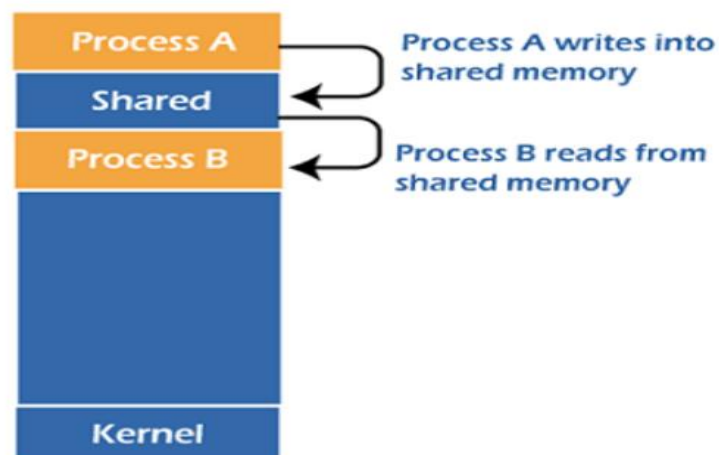
```

    exit(EXIT_FAILURE);
}
printf("Received: %s\n", msg.mtext);
// Optionally remove the message queue
if (msgctl(msgid, IPC_RMID, NULL) < 0) {
    perror("msgctl");
    exit(EXIT_FAILURE);
}
return 0;
}

```

Shared memory

Inter Process Communication through shared memory is a concept where two or more process can access the common memory and communication is done via this shared memory where changes made by one process can be viewed by another process.



The problem with pipes, fifo and message queue – is that for two process to exchange information. The information has to go through the kernel.

- Server reads from the input file.
- The server writes this data in a message using either a pipe, fifo or message queue.
- The client reads the data from the IPC channel, again requiring the data to be copied from kernel's IPC buffer to the client's buffer.
- Finally, the data is copied from the client's buffer.

A total of four copies of data are required (2 read and 2 write). So, shared memory provides a way by letting two or more processes share a memory segment. With Shared Memory the data is only copied twice – from input file into shared memory and from shared memory to the output file.

shmget: This function is used to allocate a shared memory segment.

Syntax: `int shmget(key_t key, size_t size, int shmflg);`

- **key_t key:** Unique identifier for the shared memory segment.
- **size_t size:** The size of the shared memory segment in bytes.
- **int shmflg:** Permission flags and creation flags (**IPC_CREAT**, **IPC_EXCL**).

Returns: On success, it returns the identifier of the shared memory segment; on error, it returns -1

Shmat : This function attaches the shared memory segment identified by the shared memory ID to the address space of the calling process.

Syntax : `void *shmat(int shmid, const void *shmaddr, int shmflg);`

- **nt shmid:** Shared memory identifier returned by **shmget**.
- **const void shmaddr:* Suggested starting address for the attachment; usually set to **NULL** to let the system choose.
- **int shmflg:** Flags for the operation, typically **0** or **SHM_RDONLY** for read-only access.

Returns: On success, it returns the address of the attached shared memory segment; on failure, it returns **(void *) -1**.

Shmdt: This function detaches the shared memory segment from the address space of the calling process.

Syntax: `int shmdt(const void *shmaddr);`

- **const void shmaddr:* Address of the shared memory segment to detach.

Returns: On success, returns 0; on error, returns -1.

Shmctl: This function performs various control operations on the shared memory segment.

Syntax: `int shmctl(int shmid, int cmd, struct shmid_ds *buf);`

- **int shmid:** Shared memory identifier.
- **int cmd:** Control command (**IPC_STAT**, **IPC_SET**, **IPC_RMID**).
- **struct shmid_ds buf:* Pointer to a **shmid_ds** structure to store or modify shared memory metadata.

```

struct shmid_ds {
    struct ipc_perm shm_perm; // Operation permission structure
    size_t      shm_segsz; // Size of segment in bytes
    time_t      shm_atime; // Last attach time
    time_t      shm_dtime; // Last detach time
    time_t      shm_ctime; // Last change time
    pid_t       shm_cpid; // PID of creator
    pid_t       shm_lpid; // PID of last operation
    shmatt_t     shm_nattch; // Number of current attaches
};

```

Return Value: On success, returns 0; on error, returns -1.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>

int main() {
    key_t key = 1234; // Identifier for the shared memory segment
    int shmid;
    char *data;

    // Create the shared memory segment
    shmid = shmget(key, 1024, 0666 | IPC_CREAT);
    if (shmid < 0) {
        perror("shmget");
        exit(1);
    }

    // Attach the shared memory segment
    data = shmat(shmid, NULL, 0);
    if (data == (char *)(-1)) {
        perror("shmat");
        exit(1);
    }
}

```

```

}

// Write to the shared memory
printf("Writing to shared memory: \"Hello, World!\\n\\n");
strncpy(data, "Hello, World!", 1024);

// Detach the shared memory
if (shmdt(data) < 0) {
    perror("shmdt");
    exit(1);
}

// Reattach the shared memory to read from it
data = shmat(shmid, NULL, 0);
if (data == (char *)(-1)) {
    perror("shmat");
    exit(1);
}

// Read from the shared memory
printf("Reading from shared memory: \"%s\\n\\n\", data);

// Detach the shared memory
if (shmdt(data) < 0) {
    perror("shmdt");
    exit(1);
}

// Remove the shared memory segment
if (shmctl(shmid, IPC_RMID, NULL) < 0) {
    perror("shmctl");
    exit(1);
}

return 0;}

```

ipc status commands

Interprocess Communication (IPC) mechanisms in Unix-like operating systems, such as semaphores, message queues, and shared memory, offer status commands to help manage and monitor IPC resources. These status commands are generally invoked using control system calls like **semctl**, **msgctl**, and **shmctl**, particularly with the **IPC_STAT** command.

1. Shared Memory (shmctl with IPC_STAT)

shmctl with the **IPC_STAT** command is used to fetch the status of a shared memory segment.

Syntax: `int shmctl(int shmid, int cmd, struct shmid_ds *buf);`

- **shmid:** Identifier for the shared memory segment.
- **cmd:** Command, where **IPC_STAT** is used to get the status.
- **buf:** Pointer to **shmid_ds** structure where the status data will be stored.

Data Fetched Includes:

- Segment size, owner, permissions
- Time of last attach, detach, and changes
- PIDs of creator and last operator
- Number of current attachments

2. Message Queues (msgctl with IPC_STAT)

msgctl with the **IPC_STAT** command retrieves the status of a message queue.

Syntax: `int msgctl(int msqid, int cmd, struct msqid_ds *buf);`

- **msqid:** Identifier for the message queue.
- **cmd:** Command, where **IPC_STAT** retrieves the current status.
- **buf:** Pointer to **msqid_ds** structure to store the status data.

Data Fetched Includes:

- Queue size, owner, permissions
- Time of last send and receive operations
- Number of messages currently in the queue
- Total bytes allowed in the queue
- PIDs of processes that last sent or received a message

3. Semaphores (semctl with IPC_STAT)

semctl with the **IPC_STAT** command is used to obtain the status of a semaphore set.

Syntax: `int semctl(int semid, int semnum, int cmd, union semun arg);`

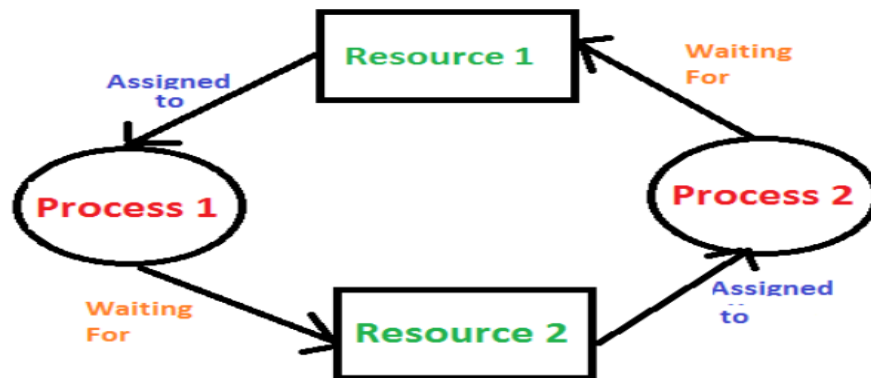
- **semid**: Identifier for the semaphore set.
- **semnum**: Semaphore number within the set (usually ignored for **IPC_STAT**).
- **cmd**: Command, with **IPC_STAT** used for fetching status.
- **arg**: A union where one member is a pointer to **semid_ds**.

Data Fetched Includes:

- Semaphore array sizes, owner, permissions
- Time of last operations
- Number of semaphores in the set

Deadlocks

In a multiprogramming system, multiple processes compete for limited resources. When a process requests resources that are unavailable at the time, it goes into a waiting state. Sometimes, a waiting process cannot change its state again because other processes hold the resources it needs, resulting in a deadlock. A **deadlock** is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.



System Model

A system model or structure comprises a fixed number of resources that must be shared among different processes. These resources are divided into different types, each having a specific number of identical instances. Some examples of resource types are memory space, CPU cycles, directories and files, and I/O devices such as keyboards, printers, and CD/DVD drives. For instance, when a system has two CPUs, the resource type CPU has two instances.

Under the standard mode of operation, any process may use a resource in only the below-mentioned sequence:

1. **Request:** When the request can't be approved immediately (where the case may be when another process is utilizing the resource), the requesting job must wait until it can obtain the resource.
2. **Use:** The process can run on the resource (like when the resource is a printer, its job/process is to print on the printer).
3. **Release:** The process releases the resource (like terminating or exiting any specific process).

Deadlock Characterization

Necessary conditions

Deadlocks can arise if the following four conditions hold simultaneously in a system:

- Mutual exclusion: If there is a non-shareable resource, only one process can use a resource at a time.
- Hold and wait: A process holding at least one resource is waiting to acquire additional resources held by other waiting processes.
- No preemption: A resource can be released only voluntarily by the process that is holding it after the process has completed its task. It is not possible to preempt a resource forcibly from a process.
- Circular wait: If there exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0 .

In order to describe deadlocks in a more precise way directed graphs are used that are called **system**

Resource Allocation Graph.

- This Graph acts as the pictorial representation of the state of the system.
- The Resource Allocation graph mainly consists of a **set of vertices V** and a **set of Edges E**.
- This graph mainly contains all the information related to the processes that are holding some resources and also contains the information of the processes that are waiting for some more resources in the system.
- Also, this graph contains all the information that is related to all the instances of the resources which means the information about available resources and the resources which are being used by the process

Components of Resource Allocation Graph

Given below are the components of RAG:

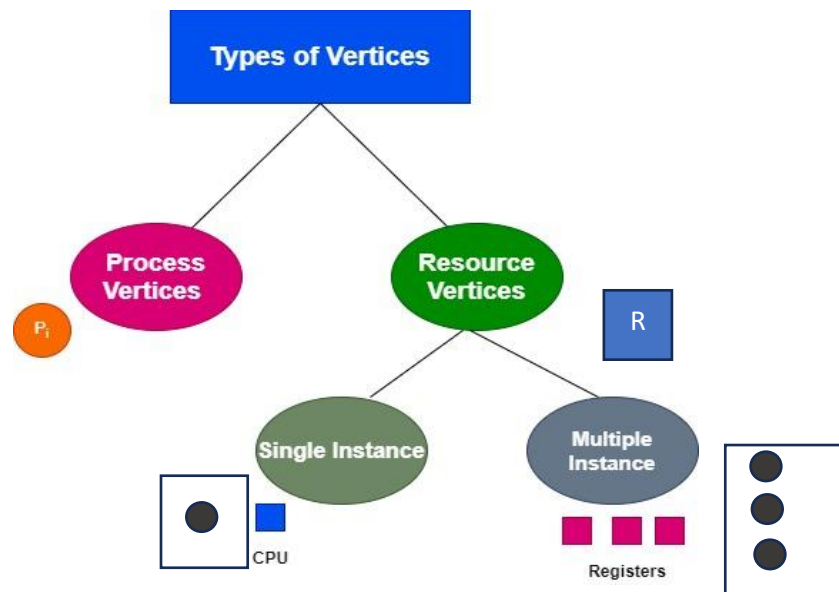
1. Vertices
2. Edges

Vertices

There are two kinds of vertices used in the resource allocation graph and these are:

Process Vertices : These vertices are used in order to represent process vertices. The circle is used in order to draw the process vertices and the name of the process is mentioned inside the circle.

Resource Vertices: These vertices are used in order to represent resource vertices. The rectangle is used in order to draw the resource vertices and we use dots inside the circle to mention the number of instances of that resource.



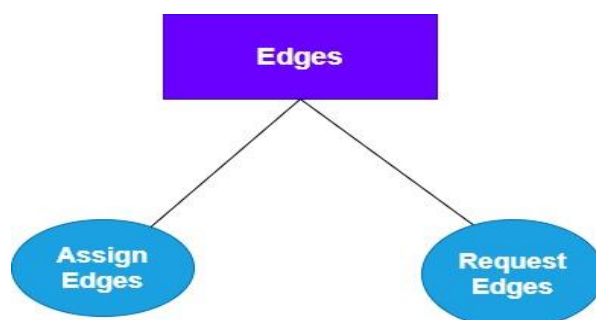
In the system, there may exist a number of instances and according to them, there are two types of resource vertices and these are single instances and multiple instances.

Single Instance: In a single instance resource type, there is a single dot inside the box. The single dot mainly indicates that there is one instance of the resource.

Multiple Instance: In multiple instance resource types, there are multiple dots inside the box, and these Multiple dots indicate that there are multiple instances of the resources.

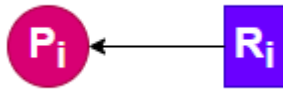
Edges

In the Resource Allocation Graph, Edges are further categorized into two:



1. Assign Edges

Assign Edges are mainly used to represent the allocation of resources to the process. We can draw assign edges with the help of an arrow in which mainly the arrowhead points to the process, and the tail points to the instance of the resource.



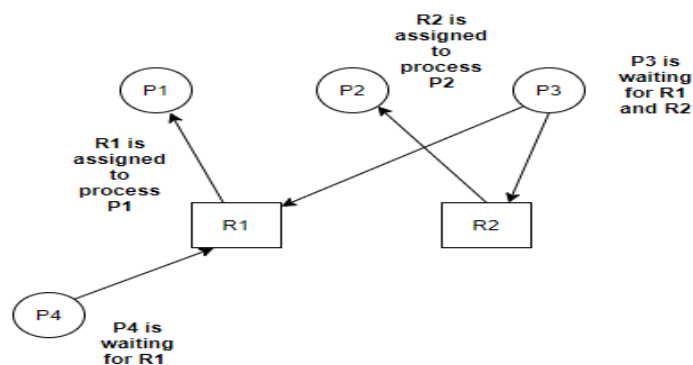
2. Request Edges

Request Edge is mainly used to signify the waiting state of the process. Likewise in assigned edge, an arrow is used to draw an arrow edge. But Here the arrowhead points to the instance of a resource, and the tail points to the process.

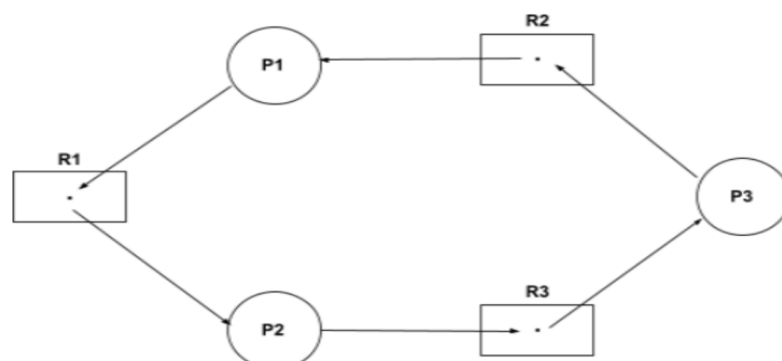


Single Instance RAG : If there is a cycle in the Resource Allocation Graph and each resource in the cycle provides only one instance, then the processes will be in deadlock.

Suppose there are Four Processes P1, P2, P3, P4, and two resources R1 and R2, where P1 is holding R1 and P2 is holding R2, P3 is waiting for R1 and R2 while P4 is waiting for resource R1.



In the above example, there is no circular dependency so there are no chances for the occurrence of deadlock.



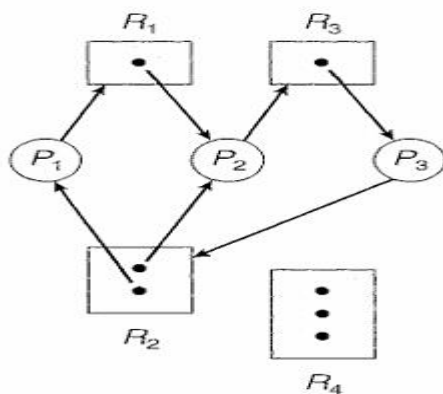
In the above single instances RAG example, it can be seen that P2 is holding the R1 and waiting for R3. P1 is waiting for R1 and holding R2 and, P3 is holding the R3 and waiting for R2. So, it can be concluded that none of the processes will get executed. It leads to Deadlock

Multiple Instance RAG Example

If each resource type has several instances, then a cycle does not necessarily imply that a deadlock has occurred. In this case, a cycle in the graph is a necessary but not a sufficient condition for the existence of deadlock.

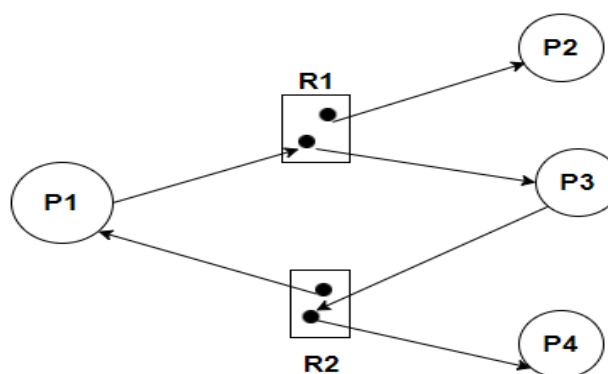
suppose that process P3 requests an instance of resource type R2. Since no resource instance is currently available, a request edge $P_3 \rightarrow R_2$ is added to the graph (At this point, two minimal cycles exist in the system: $P_1 \rightarrow R_1 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$

$P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_2$



Processes P1, P2, and P3 are deadlocked. Process P2 is waiting for the resource R3, which is held by process P3. Process P3 is waiting for either process P1 or process P2 to release resource R2. In addition, process P1 is waiting for process P2 to release resource R1.

Suppose there are four processes P1, P2, P3, P4 and there are two instances of resource R1 and two instances of resource R2:



Multiple Instance Resource Allocation graph with a cycle but no deadlock

One instance of R2 is assigned to process P1 and another instance of R2 is assigned to process P4, Process P1 is waiting for resource R1.

One instance of R1 is assigned to Process P2 while another instance of R2 is assigned to process P3, Process P3 is waiting for resource R2. However, there is no deadlock. Observe that process P4 may release its instance of resource type R2. That resource can then be allocated to P3, breaking the cycle.

In summary, if a resource-allocation graph does not have a cycle, then the system is not in a deadlocked state.

If each resource type has exactly one instance, then a cycle implies that a deadlock has occurred. In this case, a cycle in the graph is both a necessary and a sufficient condition for the existence of deadlock.

If each resource type has several instances, then a cycle does not necessarily imply that a deadlock has occurred. In this case, a cycle in the graph is a necessary but not a sufficient condition for the existence of deadlock.

Deadlock Vs Starvation

Deadlock	Starvation
A deadlock is a condition in operating systems in which no process proceeds for execution and wait for resources that have been acquired by some other processes. It is also known as circular wait.	Starvation is a situation when a process keeps waiting (and starving) for a resource that is being held by other highpriority processes.
In a deadlock, all the involved processes keep waiting for each other to get completed.	In this situation, the high priority processes are executed, and low priority processes are blocked.
In a deadlock, none of them can execute because they are waiting for the other process to complete.	In starvation, the low priority process starves due to lack of resources.
The resources are blocked by the process.	Resources are utilized by high priority process continuously.
The necessary conditions for deadlock are mutual exclusion, hold and wait, no pre-emption, and circular wait.	In starvation, priorities are assigned to process.
Deadlock can be prevented by avoiding conditions such as mutual exclusion, hold and wait, no preemption and circular wait.	Starvation can be prevented using the "Aging" technique.

Methods for Handling Deadlocks

To handle deadlock, there are several techniques that can be used in the operating system. Some of the most common deadlock handling techniques are:

1. **Prevention:** This technique involves designing the system in such a way that deadlock can never occur. This is achieved by ensuring that at least one of the four necessary conditions for deadlock (mutual exclusion, hold and wait, no pre-emption, and circular wait) does not occur. Prevention can be expensive and can reduce system performance, so it is not always the best option.
2. **Avoidance:** This technique involves ensuring that the system does not enter a deadlock state by using algorithms that can predict if a resource allocation request will result in a deadlock. The system will only grant resource requests that do not lead to a deadlock state. This technique requires a lot of computation and can be resource-intensive.
3. **Detection and Recovery:** This technique involves detecting when a deadlock has occurred and taking steps to recover from it. One common approach is to use an algorithm that periodically checks for deadlock. If a deadlock is detected, the system will then recover from it by releasing resources or killing processes.
4. **Ignoring Deadlock:** In some cases, it may be less expensive to simply ignore deadlock and let the processes remain blocked until the system is restarted or the resources are manually released.

Deadlock Prevention

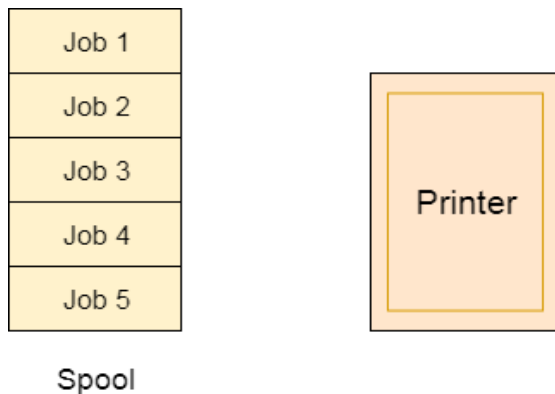
Let us take an example of a chair, as we know that chair always stands on its four legs. Likewise, for the deadlock problem, all the above given four conditions are needed. If anyone leg of the chair gets broken, then definitely it will fall. The same is the situation with the deadlock if we become able to violate any condition among the four and do not let them occur together then there can be prevented from the deadlock problem.

Mutual Exclusion

This condition must hold for non-sharable resources. In contrast, Sharable resources do not require mutually exclusive access and thus cannot be involved in a deadlock. A good example of a sharable resource is Read-only files because if several processes attempt to open a read-only file at the same time, then they can be granted simultaneous access to the file.

A process need not to wait for the sharable resource. Spooling can be an effective approach to violate mutual exclusion .

For a device like printer, spooling can work. There is a memory associated with the printer which stores jobs from each of the process into it. Later, Printer collects all the jobs and print each one of them according to FCFS. By using this mechanism, the process doesn't have to wait for the printer and it can continue whatever it was doing. Later, it collects the output when it is produced.



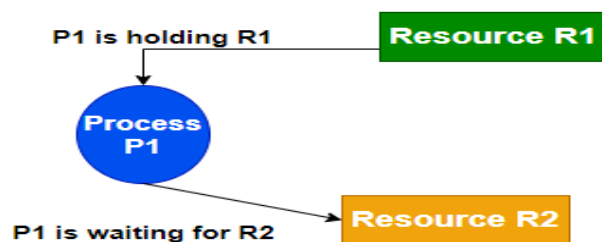
Although, Spooling can be an effective approach to violate mutual exclusion but it suffers from two kinds of problems.

1. This cannot be applied to every resource.
2. After some point of time, there may arise a race condition between the processes to get space in that spool.

We cannot force a resource to be used by more than one process at the same time since it will not be fair enough and some serious problems may arise in the performance. Therefore, we cannot violate mutual exclusion for a process practically.

Hold and Wait

Hold and wait condition occurs when a process holds a resource and is also waiting for some other resource in order to complete its execution. Thus, if we did not want the occurrence of this condition then we must guarantee that when a process requests a resource, it does not hold any other resource.



Hold and wait condition

There are some protocols that can be used in order to ensure that the Hold and Wait condition never occurs:

- According to the first protocol; Each process must request and gets all its resources before the beginning of its execution.
- The second protocol allows a process to request resources only when it does not occupy any resource.

Let us illustrate the difference between these two protocols:

We will consider a process that mainly copies data from a DVD drive to a file on disk, sorts the file, and then prints the results to a printer. If all the resources must be requested at the beginning of the process according to the first protocol, then the process requests the DVD drive, disk file, and printer initially. It will hold the printer during its entire execution, even though the printer is needed only at the end.

While the second method allows the process to request initially only the DVD drive and disk file. It copies the data from the DVD drive to the disk and then releases both the DVD drive and the disk file. The process must then again request the disk file and printer. After copying the disk file to the printer, the process releases these two resources as well and then terminates.

Disadvantages of Both Protocols

- Utilization of resources may be low, since resources may be allocated but unused for a long period. In the above-given example, for instance, we can release the DVD drive and disk file and again request the disk file and printer only if we can be sure that our data will remain on the disk file. Otherwise, we must request all the resources at the beginning of both protocols.
- There is a possibility of starvation. A process that needs several popular resources may have to wait indefinitely because at least one of the resources that it needs is always allocated to some other process.

No Preemption

The third necessary condition for deadlocks is that there should be no preemption of resources that have already been allocated. In order to ensure that this condition does not hold the following protocols can be used :

- According to the First Protocol: "If a process that is already holding some resources requests another resource and if the requested resources cannot be allocated to it, then it must release all the resources currently allocated to it."

- According to the Second Protocol: "When a process requests some resources, if they are available, then allocate them. If in case the requested resource is not available then we will check whether it is being used or is allocated to some other process waiting for other resources. If that resource is not being used, then the operating system preempts it from the waiting process and allocate it to the requesting process. And if that resource is being used, then the requesting process must wait".
The second protocol can be applied to those resources whose state can be easily saved and restored later for example CPU registers and memory space, and cannot be applied to resources like printers and tape drivers.

Circular Wait

The Fourth necessary condition to cause deadlock is circular wait, In order to ensure violate this condition we can do the following:

Assign a priority number to each resource. There will be a condition that any process cannot request for a lesser priority resource. This method ensures that not a single process can request a resource that is being utilized by any other process and due to which no cycle will be formed.

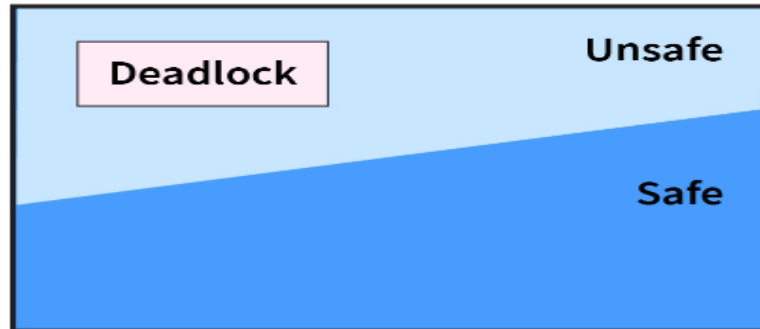
Example: Assume that R5 resource is allocated to P1, if next time P1 asks for R4, R3 that are lesser than R5; then such request will not be granted. Only the request for resources that are more than R5 will be granted.

S.No	Necessary Conditions	Approach	Practical Implementation
1	Mutual Exclusion	The approach used to violate this condition is spooling.	Not possible
2	Hold and wait	In order to violate this condition, the approach is to request all the resources for a process initially	Not possible
3	No Preemption	In order to violate this condition, the approach is snatch all the resources from the process.	Not possible
4	Circular Wait	In this approach is to assign priority to each resource and order them numerically	possible

Deadlock Avoidance

Deadlock Avoidance work by letting the Operating System know the Process requirements of resources to complete their execution, and accordingly operating system checks if the requirements can be satisfied or not.

Safe State and Unsafe State



A safe state refers to a system state where the allocation of resources to each process ensures the avoidance of deadlock. The successful execution of all processes is achievable, and the likelihood of a deadlock is low. The system attains a safe state when a suitable sequence of resource allocation enables the successful completion of all processes.

Conversely, an unsafe state implies a system state where a deadlock may occur. The successful completion of all processes is not assured, and the risk of deadlock is high. The system is insecure when no sequence of resource allocation ensures the successful execution of all processes.

Example

- To illustrate, we consider a system with twelve magnetic tape drives and three processes: P₀, P₁, and P₂. Process P₀ requires ten tape drives, process P₁ may need as many as four tape drives, and process P₂ may need up to nine tape drives. Suppose that, at time t₀, process P₀ is holding five tape drives, process P₁ is holding two tape drives, and process P₂ is holding two tape drives. (Thus, there are three free tape drives.)

	<u>Maximum Needs</u>	<u>Current Needs</u>	<u>maximum need to finished</u>
P ₀	10	5	5
P ₁	4	2	2
P ₂	9	2	7

- At time t₀, the system is in a safe state. The sequence satisfies the safety condition. Process P₁ can immediately be allocated all its tape drives and then return them (the system will then have five available tape drives); then process P₀ can get all its tape drives and return them (the system will then have ten available tape drives); and finally process P₂ can get all its tape drives and return them (the system will then have all twelve tape drives available).

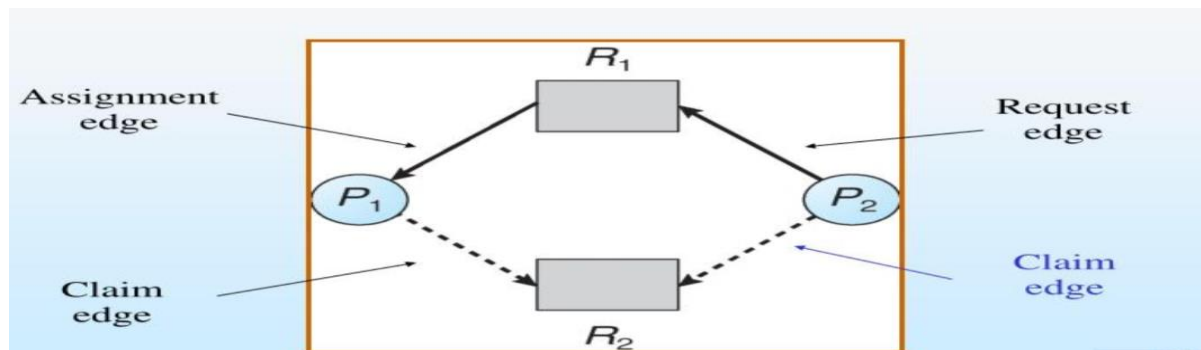
- A system can go from a safe state to an unsafe state. Suppose that, at time t_1 , process P_2 requests and is allocated one more tape drive. The system is no longer in a safe state. At this point, only process P_1 can be allocated all its tape drives. When it returns them, the system will have only four available tape drives. Since process P_0 is allocated five tape drives but has a maximum of ten, it may request five more tape drives. If it does so, it will have to wait, because they are unavailable. Similarly, process P_2 may request six additional tape drives and have to wait, resulting in a deadlock. Our mistake was in granting the request from process P_2 for one more tape drive. If we had made P_2 wait until either of the other processes had finished and released its resources, then we could have avoided the deadlock.

Deadlock Avoidance Algorithms

- When resource categories have only single instances of their resources, Resource- Allocation Graph Algorithm is used. In this algorithm, a cycle is a necessary and sufficient condition for deadlock.
- When resource categories have multiple instances of their resources, Banker's Algorithm is used. In this algorithm, a cycle is a necessary but not a sufficient condition for deadlock.

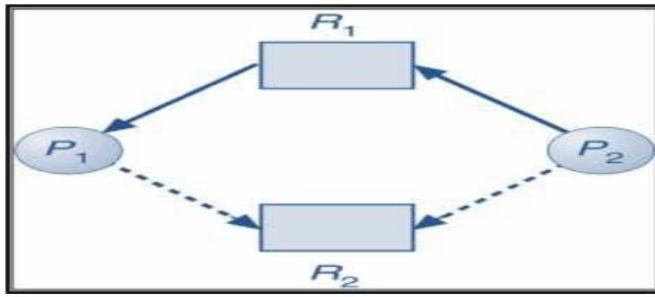
Resource Allocation graph

Resource Allocation graph technique is used for deadlock avoidance when there is a single instance of every resource. In resource allocation graph for deadlock avoidance we introduce a third kind of edge called the claim edge which is a dotted line from a process towards a resource meaning that the resource can be requested by the process in future.

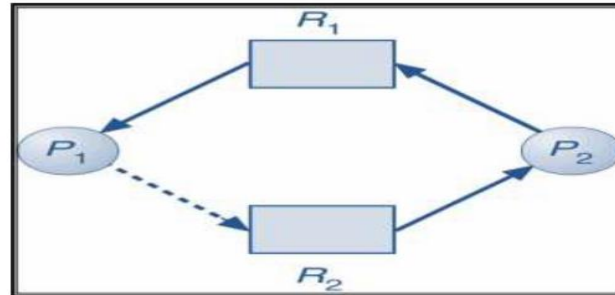


- Whenever a process requests for a resource the claim edge is changed to request edge and
- if the resource can be granted the request edge is changed to assignment edge.
- After this change look for a cycle in the graph.
- If no cycle exists, then the system is in safe state and the deadlock will not occur
- else the system is in unsafe state and deadlock may or may not occur.

Consider the resource-allocation graph in Figure below. P_2 requests for resource R_2 as shown using the claim edge from P_2 to R_2 .



This is converted to an assignment edge in the resource-allocation graph as shown in Figure .



The graph is now checked for cycles. Since there is a cycle, the system understands that allocating R_2 to P_2 will lead to a deadlock. The only instance of R_1 is assigned to P_1 and the only instance of R_2 is assigned to P_2 . P_2 has requested for R_1 . It is known from the claim edge that process P_1 may request for R_2 . If P_1 requests for R_2 (before releasing R_1 , then both the processes P_1 and P_2 will wait for a resource held by the other process. The system will go to a deadlocked state. Therefore, R_2 is not allocated to P_2 . Thus, deadlocks are avoided.

This method can be used to avoid deadlocks only if there is a single instance of each resource type. If there are more instances of each resource type, Banker's algorithm is used for deadlock avoidance.

Bankers Algorithm

Banker's algorithm is a **deadlock avoidance algorithm**. It is named so because this algorithm is used in banking systems to determine whether a loan can be granted or not.

When working with a banker's algorithm, it requests to know about three things:

1. How much each process can request for each resource in the system. It is denoted by the [**MAX**] request.
2. How much each process is currently holding each resource in a system. It is denoted by the [**ALLOCATED**] resource.
3. It represents the number of each resource currently available in the system. It is denoted by the [**AVAILABLE**] resource.

Following are the important data structures terms applied in the banker's algorithm as follows. Suppose n is the number of processes, and m is the number of each type of resource used in a computer system.

1. **Available:** It is an array of length ' m ' that defines each type of resource available in the system. When $\text{Available}[j] = K$, means that ' K ' instances of Resources type $R[j]$ are available in the system.

2. **Max:** It is a $[n \times m]$ matrix that indicates each process $P[i]$ can store the maximum number of resources $R[j]$ (each type) in a system.
3. **Allocation:** It is a matrix of $m \times n$ orders that indicates the type of resources currently allocated to each process in the system. When $\text{Allocation}[i, j] = K$, it means that process $P[i]$ is currently allocated K instances of Resources type $R[j]$ in the system.
4. **Need:** It is an $M \times N$ matrix sequence representing the number of remaining resources for each process. When the $\text{Need}[i][j] = k$, then process $P[i]$ may require K more instances of resources type R_j to complete the assigned work.

$$\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j].$$
5. **Finish:** It is the vector of the order m . It includes a Boolean value (true/false) indicating whether the process has been allocated to the requested resources, and all resources have been released after finishing its task.

Safety Algorithm

1. Let Work and Finish be vectors of length m and n , respectively. Initialize $\text{Work} = \text{Available}$ and $\text{Finish}[i] = \text{false}$ for $i = 0, 1, \dots, n - 1$.
2. Find an index i such that both a. $\text{Finish}[i] == \text{false}$ b. $\text{Need}_i \leq \text{Work}$ If no such i exists, go to step 4. (end of all process)
3. $\text{Work} = \text{Work} + \text{Allocation}_i$ $\text{Finish}[i] = \text{true}$ Go to step 2.
4. If $\text{Finish}[i] == \text{true}$ for all i , then the system is in a safe state.

This algorithm may require an order of $m \times n^2$ operations to determine whether a state is safe.

Example:

Let us consider the following snapshot for understanding the banker's algorithm:

Processes	Allocation A B C	Max A B C	Available A B C
P0	1 1 2	4 3 3	2 1 0
P1	2 1 2	3 2 2	

P2	4 0 1	9 0 2	
P3	0 2 0	7 5 3	
P4	1 1 2	1 1 2	

1. calculate the content of the need matrix?
2. Check if the system is in a safe state?
3. Determine the total sum of each type of resource?

Solution:

1. The Content of the need matrix can be calculated by using the formula given below:

Need = Max – Allocation

Process	Need		
	A	B	C
P₀	3	2	1
P₁	1	1	0
P₂	5	0	1
P₃	7	3	3
P₄	0	0	0

2. Let us now check for the safe state.

Safe sequence:

1. For process P₀, Need = (3, 2, 1) and
work = (2, 1, 0)
Need <= work = False
So, the system will move to the next process.

2. For Process P₁, Need = (1, 1, 0)

Available = (2, 1, 0)
Need <= Available = True
Request of P₁ is granted.
work = work + Allocation
= (2, 1, 0) + (2, 1, 2)
= (4, 2, 2) (New work)

3. For Process P₂, Need = (5, 0, 1)

work = (4, 2, 2)
Need <= work = False
So, the system will move to the next process.

4. For Process P3, Need = (7, 3, 3)

work = (4, 2, 2)

Need <= work = False

So, the system will move to the next process.

5. For Process P4, Need = (0, 0, 0)

work = (4, 2, 2)

Need <= work = True

Request of P4 is granted.

work = work + Allocation

= (4, 2, 2) + (1, 1, 2)

= (5, 3, 4) now, (New work)

6. Now again check for Process P2, Need = (5, 0, 1)

work = (5, 3, 4)

Need <= work = True

Request of P2 is granted.

work = work + Allocation

= (5, 3, 4) + (4, 0, 1)

= (9, 3, 5) now, (New work)

7. Now again check for Process P3, Need = (7, 3, 3)

work = (9, 3, 5)

Need <= work = True

The request for P3 is granted.

work = work + Allocation

= (9, 3, 5) + (0, 2, 0) = (9, 5, 5)

8. Now again check for Process P0, = Need (3, 2, 1)

work (9, 5, 5)

Need <= work = True

So, the request will be granted to P0.

Safe sequence: < P1, P4, P2, P3, P0 >

The system allocates all the needed resources to each process. So, we can say that the system is in a safe state.

The total amount of resources will be calculated by the following formula:

The total amount of resources = sum of columns of allocation + Available

= [8 5 7] + [2 1 0] = [10 6 7]

Resource-Request Algorithm

Next, we describe the algorithm for determining whether requests can be safely granted.

Let Request_i be the request vector for process P_i. If Request_i [j] == k, then process P_i wants k instances of resource type R_j. When a request for resources is made by process P_i, the following actions are taken:

1. If Request_i ≤ Need_i, go to step 2. Otherwise, raise an error condition, since the process has exceeded its maximum claim.
2. If Request_i ≤ Available, go to step 3. Otherwise, P_i must wait, since the resources are not available.

3. Have the system pretend (offering) to have allocated the requested resources to process P_i by modifying the state as follows:
 $Available = Available - Request_i$;
 $Allocation_i = Allocation_i + Request_i$;
 $Need_i = Need_i - Request_i$;
4. Apply safety algorithm. If the resulting resource-allocation state is safe, the transaction is completed, and process P_i is allocated its resources. However, if the new state is unsafe, then P_i must wait for $Request_i$, and the old resource allocation state is restored.

Example

If the system is in a safe state, can the following requests be granted, why or why not? Please also run the safety algorithm on each request as necessary.

- a. P_1 requests (2,1,1,0)

We cannot grant this request, because we do not have enough available instances of resource A.

- b. P_1 requests (0,2,1,0)

There are enough available instances of the requested resources, so first let's pretend to accommodate the request and see the system looks like:

	Allocation				Max				Available				Need Matrix				
	A	B	C	D	A	B	C	D	A	B	C	D		A	B	C	D
P0	0	1	1	0	0	2	1	0	1	3	1	0	P ₀	0	1	0	0
P1	1	4	4	1	1	6	5	2					P ₁	0	2	1	1
P2	1	3	6	5	2	3	6	6					P ₂	1	0	0	1
P3	0	6	3	2	0	6	5	2					P ₃	0	0	2	0
P4	0	0	1	4	0	6	5	6					P ₄	0	6	4	2

Now we need to run the safety algorithm:

Initially			Let's first look at P0. Need ₀ (0,1,0,0) is less than work, so we change the work vector and finish matrix as follows:		
Work vector		Finish matrix		Work vector	
1		P ₀	False	P0	True
3		P ₁	False	P1	False
1		P ₂	False	P2	False
0		P ₃	False	P3	False
		P ₄	False	P4	False

Need1 (0,2,1,1) is not less than work, so we need to move on to P2.

Need2 (1,0,0,1) is not less than work, so we need to move on to P3.

Need3 (0,0,2,0) is less than or equal to work. Let's update work and finish:

Work vector	Finish matrix	
1	P ₀	True
10	P ₁	False
5	P ₂	False
2	P ₃	True
	P ₄	False

Let's take a look at Need4 (0,6,4,2). This is less than work, so we can update work and finish:

Work vector	Finish matrix	
1	P ₀	True
10	P ₁	False
6	P ₂	False
6	P ₃	True
	P ₄	True

We can now go back to P1. Need1 (0,2,1,1) is less than work, so work and finish can be updated:

Work vector	Finish matrix	
1	P0	True
14	P1	True
10	P2	False
7	P3	True
	P4	True

Finally, Need2 (1,0,0,1) is less than work, so we can also accommodate this. Thus, the system is in a safe state when the processes are run in the following order: P0,P3,P4,P1,P2. We therefore can grant the resource request.

Deadlock Detection & Recovery

If a system neither uses deadlock prevention nor a deadlock avoidance strategy then it might enter into a deadlock. Thus, it becomes important to use deadlock detection and recovery policy. A Deadlock detection algorithm, determines whether the system is in deadlock state or not? If yes, then the system uses a deadlock recovery method to break the deadlock.

Deadlock Detection

There are two deadlock detection methods depending upon the number of instances of each resource:

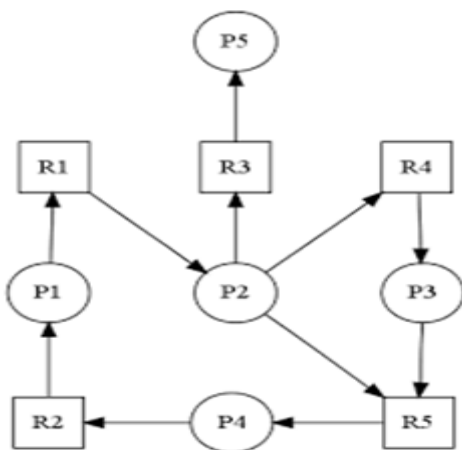
1. Single instance of each resource
2. Multiple instance of each resource

Single Instance of Each Resource: *wait-for-graph*

When there is a single instance of each resource the system can use wait-for-graph for deadlock detection. The key points about the wait-for-graph are:

1. Obtained from resource allocation graph
2. Remove the resource nodes – from the resource allocation graph remove the resource nodes
3. Collapse the corresponding edges

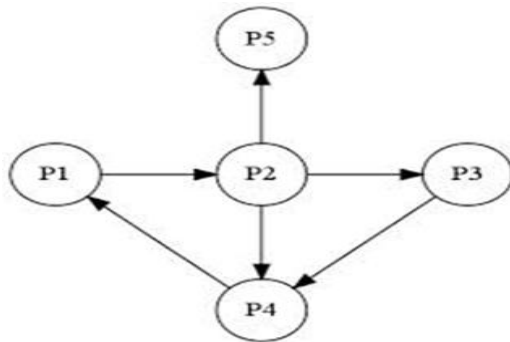
Resource Allocation Graph



An Edge from P_i to P_j means that process P_i is waiting for a resource held by process P_j . Now, if there are two edges $P_i \rightarrow R_q$ and $R_q \rightarrow P_j$ in resource allocation graph, for some resource R_q the collapse these into one single edge from $P_i \rightarrow P_j$ to make the wait-for-graph. Finally, if there is a cycle in wait-for-graph, then the system is in deadlock else not.

For example, in the above figure, P2 is requesting R3 which is held by P5. Hence, we remove the resource R3 and collapse the edge between P2 and P5. wait-for-graph reflects this change. Similarly, the collapse of edges between P4 and P1 and all other processes is done.

Wait for Graph



Multiple Instance of Each Resource

This algorithm is similar to Banker's Algorithm. (**Deadlock Detection Algorithm**)

It uses three data structures:

1. *Available*: A vector of length m indicates the number of available resources of each type.
2. *Allocation*: An $n \times m$ matrix defines the number of resources of each type currently allocated to each process
3. *Request*: An $n \times m$ matrix indicates the current request of each process. For example, if $Request[ij] = k$, then process P_i is requesting k more instances of the resource type R_j .

Deadlock Detection Algorithm

1. Let *Work* and *Finish* be vectors of length m and n , respectively

Initialize:

(a) $Work = Available$

(b) For $i = 1, 2, \dots, n$, if $Allocation_i \neq 0$, then

$Finish[i] = false$; otherwise, $Finish[i] = true$.

2. Find an index i such that both:

(a) $Finish[i] == false$

(b) $Request_i \leq Work$

If no such i exists, go to step 4.

3. **$Work = Work + Allocation_i$**

$Finish[i] = true$

go to step 2.

4. If $Finish[i] == false$, for some i , $1 \leq i \leq n$, then the system is in deadlock state.

Moreover, if $Finish[i] == false$, then P_i is deadlocked.

Algorithm requires an order of $O(m \times n^2)$ operations to detect whether the system is in deadlocked state.

Example 1 of deadlock detection

Consider 5 processes P_0 through P_4 ; 3 resource types A , B and C . There are 7 instances of A , 2 instances of B and 6 instances of C .

The snapshot at time T_0 is given below:

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	$A \ B \ C$	$A \ B \ C$	$A \ B \ C$
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

We now simulate the algorithm for the above example. Initially, $work = (0,0,0)$; $Finish[i] = false$ for $i = 0,1,2,3,4$ $i = 0$

We check if $Request_0 \leq work$? Yes

Therefore, $Work = Work + Allocation_0 = (0,0,0) + (0,1,0) = (0,1,0)$ $Finish[0] = true$, P_0 added to safe sequence $\langle P_0 \rangle$

$Work = (0,1,0)$;

Is $Request_1 \leq work$? No

Since $Request_1$ is not less than Available, check the next process.

$Work = (0,1,0)$;

Is $Request_2 \leq work$? Yes

$Work = Work + Allocation_2 = (0,1,0) + (3,0,3) = (3,1,3)$

$Finish[2] = true$, P_2 added to safe sequence $\langle P_0, P_2 \rangle$

Work = (3,1,3);

Is Request3 ≤ work? Yes

Work = Work + Allocation3 =(3,1,3) + (2,1,1) = (5,2,4) Finish[3] = true,

P_3 added to safe sequence and the safe sequence is now $\langle P_0, P_2, P_3 \rangle$

Work = (5,2,4);

Is Request4 ≤ work? Yes

Work = Work + Allocation4 =(5,2,4) + (0,0,2) = (5,2,6) Finish[4] = true,

P_4 added to safe sequence and the safe sequence is $\langle P_0, P_2, P_3, P_4 \rangle$

Now, we check again from the beginning all the other processes that were not added to the safe sequence.

Work = (5,2,6);

Is Request1 ≤ work? Yes

Work = Work + Allocation1 =(5,2,6) + (2,0,0) = (7,2,6)

Finish[1] = true,

P_1 added to safe sequence and the safe sequence now is $\langle P_0, P_2, P_3, P_4, P_1 \rangle$

Sequence $\langle P_0, P_2, P_3, P_4, P_1 \rangle$ now results in $Finish[i] = \text{true}$ for all i .

There can be more than one safe sequence, that is there can be correct safe sequences other than $\langle P_0, P_2, P_3, P_1, P_4 \rangle$. We have found one safe sequence. Since there is at least one safe sequence, the system is in a safe state. There is no deadlock in the system.

Example 2 of deadlock detection

Let process P_2 now make an additional request for an instance of resource type C. The Request matrix is changed as shown below, after including the request of an instance of resource type C by process P_2 .

	<u>Request</u>		
	A	B	C
P_0	0	0	0
P_1	2	0	2

P2 0 0 1

P3 1 0 0

P4 0 0 2

Now, let us check if the system will be in a safe state. The deadlock detection algorithm is run again.

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	<i>A B C</i>	<i>A B C</i>	<i>A B C</i>
<i>P0</i>	0 1 0	0 0 0	0 0 0
<i>P1</i>	2 0 0	2 0 2	
<i>P2</i>	3 0 3	0 0 1	
<i>P3</i>	2 1 1	1 0 0	
<i>P4</i>	0 0 2	0 0 2	

Initially, Work = Available = (0,0,0); Finish[i] = false for i = 0,1,2,3,4 When i = 0,

Check if Request₀ ≤ work? Yes

Work = Work + Allocation₀ = (0,0,0) + (0,1,0) = (0,1,0)

Finish[0] = true, *P0* is added to safe sequence < *P0* >

Work is now (0,1,0);

Is Request₁ ≤ work? No Is

Request₂ ≤ work? No Is

Request₃ ≤ work ? No Is

Request₄ ≤ work? No

Since the request of all the processes cannot be allocated, the system is not in a safe state. Though it possible to reclaim the resources held by process *P0*, there are insufficient resources to fulfill other processes' requests. Thus, a deadlock exists, consisting of processes *P1*, *P2*, *P3*, and *P4*.

2. When to invoke detection algorithm?

1. Every time a process requests for a resource

- Advantages

- It is easy to identify the processes involved in the deadlock
 - Also, the process which caused the deadlock is known. This is possible because the system invokes the algorithm after each request.
 - Disadvantage: Overhead in computation time
2. After fixed-interval of time
 - Disadvantage: Can not tell which process caused the deadlock

3. Deadlock Recover

If a deadlock is detected in the system, the next step is to recover from it. There are two methods to do so:

1. Process termination
2. Preempt resources from some of the deadlocked processes

3.1 Process Termination

1. Terminate all the deadlocked processes.
2. Terminate processes one by one until the deadlock is broken. However, the issue is to decide which process to terminate. Certain factors that can be used are:
 - How long the process has computed?
 - How much the process has finished its working?
 - What is the priority of the process?
 - How many resources are being used by the process?
 - How many total processes will be required to be terminated?

3.2 Resource preemption

In resource preemption some resources are preempted from certain processes and these resources can then be allocated to other process. But, certain issues that should be addressed are:

1. Select a victim – which process to select for preempting the resources.
2. Rollback – the process whose resources are preempted, can not continue execution. So, a process
 - must be rolled back to some safe state.
 - Or abort the process and restart
3. Starvation – it must be ensured that the same process does not get selected for resource preemption, as this will lead to starvation.

List of Practice Programs

1. Write a C program to create child process and allow parent process to display “parent” and the child to display “child” on the screen
2. Write a C program to Calculate average waiting time & Turn around time using FCFS Scheduling.
3. Write a C program to Calculate average waiting time & Turnaround time using SJF Scheduling.

fork()

It is the primary method of process creation on Unix-like operating systems. This function creates a new copy called the child out of the original process, that is called the parent. When the parent process closes or crashes for some reason, it also kills the child process.

Syntax:

pid=fork();

- The operating system is using a unique id for every process to keep track of all processes. And for that, fork() doesn't take any parameter and return an int value as following:
- **Zero:** if it is the child process (the process created). **Positive value:** if it is the parent process.
- The difference is that, in the parent process, fork() returns a value which represents the **process ID** of the child process. But in the child process, `fork()` returns the value 0.
- **Negative value:** if an error occurred.

First Come First Serve Scheduling

It is the easy and simple CPU Scheduling Algorithm. In this algorithm, the process which requests the CPU first, gets the CPU first for execution. It can be implemented using FIFO (First-In, First-Out) Queue method.

A simple real-life example of this algorithm is the cash counter. There is a queue on the counter. The person who arrives first at the counter receives the services first, followed by the second person, then third, and so on. The CPU process also works like this.

Advantages of FCFS:

The following are some benefits of using the FCFS scheduling algorithm:

1. The job that comes first is served first.
2. It is the CPU scheduling algorithm's simplest form.
3. It is quite easy to program.

Disadvantages

1. It is **non-pre-emptive** algorithm, which means the **process priority** doesn't matter.

If a process with very least priority is being executed, more like **daily routine backup** process, which takes more time, and all of a sudden, some other high priority process arrives, like **interrupt to avoid system crash**, the high priority process will have to wait, and hence in this case, the system will crash, just because of improper process scheduling.

2. In FCFS, the Average Waiting Time is comparatively high.

Resources utilization in parallel is not possible, which leads to **Convoy Effect**, (Convoy Effect is a situation where many processes, who need to use a resource for short time are blocked by one process holding that resource for a long time and hence poor resource(CPU, I/O etc) utilization

Shortest Job First

Till now, we were scheduling the processes according to their arrival time (in FCFS scheduling). However, SJF scheduling algorithm, schedules the processes according to their burst time. In SJF scheduling, the process with the lowest burst time, among the list of available processes in the ready queue, is going to be scheduled next.

In case of a tie, it is broken by **FCFS Scheduling**.

- Turn Around time = Exit time – Arrival time
- Waiting time = Turn Around time – Burst time

Lab Practice Programs (Ex: 2 & Ex :3)

A Process Scheduler schedules different processes to be assigned to the CPU based on particular scheduling algorithms. There are six popular process scheduling algorithms .

- First-Come, First-Served (FCFS) Scheduling
- Shortest-Job-First Scheduling
- Shortest Remaining Time
- Priority Scheduling non preemptive
- Priority Scheduling preemptive
- Round Robin(RR) Scheduling

The FCFS algorithm is the simplest of scheduling algorithms in OS. This is because the deciding principle behind it is just as its name suggests- on a first come basis. The job that requests execution first gets the CPU allocated to it, then the second, and so on.

The Shortest Job First (SJF) is a CPU scheduling algorithm that selects the shortest jobs on priority and executes them. The idea is that jobs with short burst time get done quickly, making CPU available for other, longer jobs/ processes.

Shortest remaining time (SRT) is the preemptive version of the SJF algorithm. The processor is allocated to the job closest to completion but it can be preempted by a newer ready job with shorter time to completion.

Priority scheduling is a non-preemptive algorithm and one of the most common scheduling algorithms in batch systems. Each process is assigned a priority. Process with highest priority is to be executed first and so on. Processes with same priority are executed on first come first served basis.

Preemptive Priority CPU Scheduling Algorithm is a pre-emptive method of CPU scheduling algorithm that works **based on the priority** of a process. In this algorithm, the scheduler schedules the tasks to work as per the priority, which means that a higher priority process should be executed first. In case of any conflict, i.e., when there is more than one process with equal priorities, then the pre-emptive priority CPU scheduling algorithm works on the basis of FCFS (First Come First Serve) algorithm.

Round Robin is the pre-emptive process scheduling algorithm. Each process is provided a fix time to execute, it is called a **quantum**. Once a process is executed for a given time period, it is preempted and other process executes for a given time period.

Identify the type of CPU scheduling algorithm for each of the scenario and implement the same in C Language. Calculate Average Turn Around time and Average Waiting time for each of the scheduling algorithm by taking arrival times accordingly.

Calculate the times by using gantt chart in observation and crosscheck the same with the output of the program.

1. Let's consider a school computer lab where students use computers for various educational activities, including research, programming assignments, and multimedia projects based on their arrival times. Here are some example tasks (student activities) that the lab server might receive throughout the day:

- **Student A needs to complete an online quiz:** Burst Time = 15 minutes, Arrival Time = 08:30 AM
- **Student B is working on a programming assignment:** Burst Time = 40 minutes, Arrival Time = 08:40 AM
- **Student C wants to edit a multimedia project:** Burst Time = 25 minutes, Arrival Time = 08:50 AM
- **Student D needs to research for a presentation:** Burst Time = 30 minutes, Arrival Time = 09:00 AM

Objective:

Your task as a developer is to help the students to carry their tasks by using a scheduling algorithm. Calculate the waiting time and turnaround time for each task, and determine the average waiting and turnaround time for the day's tasks.

2. Imagine an animation studio, Pixie Animations, which specializes in creating short animated films and digital commercials. Due to the studio's growing popularity and increased client demands, the animation rendering tasks have piled up. Throughout a typical day, various rendering tasks are submitted to the server. Each task has a specific duration determined by the complexity of the animation scenes involved. The goal is to minimize the overall waiting time and maximize resource utilization by strategically scheduling the tasks based on their duration. Write the Corresponding CPU Scheduling algorithm.

Here are some example tasks that the server might receive on a typical day:

- **Task A - Rendering a simple character animation:** Burst Time = 15 minutes, Arrival Time = 09:00 AM
- **Task B - Rendering a complex explosion scene:** Burst Time = 30 minutes, Arrival Time = 09:10 AM
- **Task C - Rendering a detailed background scene:** Burst Time = 20 minutes, Arrival Time = 09:15 AM

- **Task D - Rendering a short dialogue scene:** Burst Time = 10 minutes, Arrival Time = 09:30 AM

3. Imagine an Emergency Medical Dispatch Center (EMDC) tasked with coordinating emergency medical responses across a large urban area. The center receives calls about various emergencies—accidents, medical crises, and urgent patient transfers. Identify the algorithm helps prioritize shorter-duration tasks, allowing the center to quickly dispatch assistance for multiple emergencies, potentially saving more lives by reducing the response time.

Task Simulation:

Here are some examples of calls received by the EMDC one morning:

1. **Call A - Rapid response for a cardiac arrest at a home:** Initial Burst Time = 7 minutes, Arrival Time = 08:00 AM
 2. **Call B - Ambulance dispatch for a minor car accident:** Initial Burst Time = 5 minutes, Arrival Time = 08:01 AM
 3. **Call C - Coordinate a helicopter for a critical transfer from a rural area:** Initial Burst Time = 3 minutes, Arrival Time = 08:02 AM
 4. **Call D - Send paramedics to a multiple injury accident on a highway:** Initial Burst Time = 20 minutes, Arrival Time = 08:15 AM
 5. **Call E - Dispatch an ambulance for a sudden stroke case:** Initial Burst Time = 10 minutes, Arrival Time = 08:20 AM
4. Airlines categorize passengers into different groups based on various criteria such as frequent flyer status, class of service, special needs, and more. Each group is assigned a priority level, and within that priority, boarding is handled in the order that passengers arrive at the gate
- **Priority Assignment:**
 1. **First Priority:** First-class passengers, high-tier frequent flyers, and passengers requiring special assistance (e.g., wheelchair users). Arrival time: 3 and assistance time: 5min
 2. **Second Priority:** Business class and mid-tier frequent flyers. Arrival time: 0 and assistance time: 10min
 3. **Third Priority:** Premium economy passengers. Arrival time: 4 and assistance time: 7min
 4. **Fourth Priority:** Economy class passengers. Arrival time: 1 and assistance time: 8min
 5. **Final Priority:** Basic economy or last-minute ticket purchasers, Arrival time: 5 and assistance time: 6min

5. An IT help desk in a large corporation handles a range of technical issues and requests from employees. Given the varied nature and urgency of these issues, which scheduling system is employed to ensure that critical IT support requests are handled promptly, while less urgent issues are addressed in an orderly fashion. IT staff can switch between tasks based on real-time reevaluation of priorities as new tickets arrive.

Considering this as the inputs: Calculate Average Turn Around time and Average Waiting time

Process Id	Priority	Arrival Time	Burst Time
1	2(L)	0	1
2	6	1	7
3	3	2	3
4	5	3	6
5	4	4	5
6	10(H)	5	15
7	9	15	8

6. A local library system incorporates several branches, each equipped with computers used for catalog searches, digital archives access, and various educational programs. During the peak hours of library operation, patrons often need to use computers for tasks like research, document processing, or accessing public records. The library staff sets up a system where each user is allotted a 2-minute slot. If a user's task is not completed within this period, they go back to the end of the queue for another turn. identify the scheduling algorithm.

Process Id	Arrival time	Burst time
P1	0	5

P2	1	3
P3	2	1
P4	3	2
P5	4	3

7. Assume you own a small community bookstore that regularly needs to update its inventory records at the end of the business day. The inventory update is divided into two main tasks: tallying sales and restocking items. Due to the store's unique cataloging system, these tasks need to be handled sequentially. Create a simple application that uses UNIX/Linux processes, to separate the tallying of sales from the restocking of items.

Expected Output:

Parent Output: "Daily sales tallied successfully."

Child Output: "Items restocked based on sales data."

Final Output by Parent: "Inventory update complete."

Exercise : 4

4a. Write a C program that illustrates communication between two unrelated processes using named pipes

4b. Write a C program that receives a message from a message queue and displays them

These two programs are to be run on a Linux terminal by creating two programs: sender and receiver for passing the data. Compile and run the programs separately on two different terminals.

The remaining three programs are to run in WSL Terminal.

4c. Write a C program to allow cooperating processes to lock a resource for exclusive use (using semaphores). Implement this for a bank account transaction system

Requirements:

1. The initial account balance is \$1000.
2. Use POSIX semaphores to synchronize access to the shared account balance.
3. Implement two functions: **deposit(int amount)** and **withdraw(int amount)**:
 - **deposit(int amount)**: Increases the account balance by the specified **amount** and prints the new balance.
 - **withdraw(int amount)**: Decreases the account balance by the specified **amount** if there are sufficient funds. If not, it prints an error message indicating insufficient funds.
 - The child process should perform a deposit & the parent process should attempt to withdraw.

Output:

Withdrawing 200 from account

New balance after withdrawal: 800

Depositing 500 to account

New balance after deposit: 1500

4d. Write a C program that illustrates the suspending and resuming process using signal

Program demonstrates creating a child process using **fork()**. The parent process should be able to:

1. Properly handle errors for **fork()** and other system calls.
 - The parent process should manage the child process by using **SIGSTOP**, **SIGCONT**, **Wait()**

Output

Parent process started
Child process is running...
Parent has suspended the child process
Parent has resumed the child process

4e. Write a C program that implements producer-Consumer system with two process using semaphore

The program uses POSIX semaphores to synchronize a producer process and a consumer process. The producer should generate a simple integer to simulate producing an item and place it in a buffer. The consumer should then read this integer from the buffer to simulate consuming the item. To simplify, the buffer can hold only one item at a time.

Requirements:

1. Use two POSIX named semaphores:
 - **semProducer**: This semaphore should indicate when the buffer is empty and it's safe for the producer to add a new item.
 - **semConsumer**: This semaphore should indicate when the buffer contains an item that needs to be consumed.
2. Manage process creation using **fork()** and ensure the parent process waits for the child process to complete before it exits.

Output

Producing item
Consuming item
Producing item
Consuming item
Producing item
Consuming item
Producing item
Consuming item
Producing item
Consuming item

Named Pipe

- Usually a named pipe appears as a file and generally processes attach to it for inter-process communication. A FIFO file is a special kind of file on the local storage which allows two or more processes to communicate with each other by reading/writing to/from this file.
- A FIFO special file is entered into the filesystem by calling *mkfifo()* in C. Once we have created a FIFO special file in this way, any process can open it for reading or writing, in the same way as an ordinary file. However, it has to be open at both ends simultaneously before you can proceed to do any input or output operations on it.

Message queue

A message queue is a linked list of messages stored within the kernel and identified by a message queue identifier. A new queue is created or an existing queue opened by **msgget()**.

System calls used for message queues:

ftok(): is use to generate a unique key.

msgget(): either returns the message queue identifier for a newly created message queue or returns the identifiers for a queue which exists with the same key value.

msgsnd(): Data is placed on to a message queue by calling msgsnd().

msgrcv(): messages are retrieved from a queue.

msgctl(): It performs various operations on a queue. Generally it is use to destroy message queue.

Semaphores

The POSIX system in Linux presents its own built-in semaphore library. To use it, we have to :

1. Include semaphore.h
2. Compile the code by linking with -lpthread -lrt

To lock a semaphore or wait we can use the **sem_wait** function:

```
int sem_wait(sem_t *sem);
```

To release or signal a semaphore, we use the **sem_post** function:

```
int sem_post(sem_t *sem);
```

A semaphore is initialised by using **sem_init**(for processes or threads) or **sem_open** (for IPC).

```
sem_init(sem_t *sem, int pshared, unsigned int value);
```

Where,

- **sem** : Specifies the semaphore to be initialized.

- **pshared** : This argument specifies whether or not the newly initialized semaphore is shared between processes or between threads. A non-zero value means the semaphore is shared between processes and a value of zero means it is shared between threads.
- **value** : Specifies the value to assign to the newly initialized semaphore.

To destroy a semaphore, we can use **sem_destroy**.

```
sem_destroy(sem_t *mutex);
```

To declare a semaphore, the data type is `sem_t`.